

TRƯỜNG ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN THỰC TẬP CHUYÊN NGÀNH
NHẬN DẠNG CHỮ SỐ VIẾT TAY
HUẤN LUYỆN VÀ SO SÁNH KNN, SVM
TRÊN TẬP DỮ LIỆU MNIST

Giảng viên hướng dẫn : THS. NGUYỄN NHỨT LAM

Sinh viên thực hiện: PHAN THÀNH THUẬN

Mã số sinh viên : 170123591

Lớp : DX23TT11

Khoá : 2023

Trà Vinh, tháng 10 năm 2026

TRƯỜNG ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN THỰC TẬP CHUYÊN NGÀNH
NHẬN DẠNG CHỮ SỐ VIẾT TAY
HUẤN LUYỆN VÀ SO SÁNH KNN, SVM
TRÊN TẬP DỮ LIỆU MNIST

Giảng viên hướng dẫn : THS. NGUYỄN NHỨT LAM

Sinh viên thực hiện: PHAN THÀNH THUẬN

Mã số sinh viên : 170123591

Lớp : DX23TT11

Khoá : 2023

Trà Vinh, tháng 10 năm 2026

TÓM TẮT ĐỒ ÁN

Đồ án huấn luyện và so sánh hiệu quả của hai mô hình học máy kinh điển — K-Nearest Neighbors (KNN) và Support Vector Machine (SVM) nhân RBF — trên bài toán nhận dạng chữ số viết tay, sử dụng bộ dữ liệu chuẩn MNIST (70.000 ảnh chữ số 0-9, kích thước 28×28 điểm ảnh mức xám). Đây là đúng đề tài đã được phân công: tìm hiểu học máy, xây dựng mô hình và ứng dụng minh họa giải quyết bài toán của đề tài.

Dữ liệu được chia theo tỉ lệ 54.000/6.000/10.000 (train/validation/test) theo phương pháp stratify (giữ nguyên tỉ lệ các chữ số ở mỗi tập). Kết quả thực nghiệm thật (không mô phỏng) trên tập test: KNN ($k=5$) đạt độ chính xác 97,08%, thời gian huấn luyện 0,07 giây; SVM (kernel RBF, $C=5$) đạt độ chính xác 98,35%, thời gian huấn luyện 235,17 giây (~3.9 phút).

SVM cho độ chính xác cao hơn KNN nhưng đổi lại thời gian huấn luyện dài hơn nhiều — thể hiện rõ sự đánh đổi giữa một bộ phân loại "lazy learner" (KNN, gần như không tốn thời gian huấn luyện vì chỉ lưu lại dữ liệu) và một bộ phân loại giải bài toán tối ưu trong lúc huấn luyện (SVM). Một chương trình demo tương tác được xây dựng bằng Streamlit, cho phép người dùng vẽ hoặc tải ảnh một chữ số lên và xem cả hai mô hình dự đoán kèm điểm tin cậy.

Từ khoá: nhận dạng chữ số viết tay, MNIST, K-Nearest Neighbors, Support Vector Machine, học máy, scikit-learn.

MỤC LỤC

LỜI MỞ ĐẦU.....	vi
1. Lý do chọn đề tài.....	vi
2. Mục tiêu nghiên cứu.....	vi
3. Đối tượng và phạm vi nghiên cứu.....	vii
4. Phương pháp nghiên cứu.....	vii
5. Cấu trúc báo cáo.....	viii
CHƯƠNG 1. TỔNG QUAN.....	1
1.1. Bài toán nhận dạng chữ số viết tay.....	1
1.2. Bộ dữ liệu MNIST.....	1
1.3. Các hướng tiếp cận.....	2
1.4. Phạm vi và lựa chọn của đồ án.....	3
1.5. Tổng kết chương.....	3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	4
2.1. Thuật toán K-Nearest Neighbors.....	4
2.2. Support Vector Machine và kernel RBF.....	5
2.3. Các độ đo đánh giá.....	6
2.4. Tổng kết chương.....	7
CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN.....	8
3.1. Quy trình tổng thể.....	8
3.2. Chuẩn bị dữ liệu.....	8
3.3. Cài đặt mô hình KNN.....	9
3.4. Cài đặt mô hình SVM.....	9
3.5. Công cụ và môi trường.....	10
3.6. Tổng kết chương.....	10
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	11
4.1. Cách chia dữ liệu.....	11
4.2. Kết quả tổng thể trên tập test.....	11
4.3. Kết quả phân theo từng chữ số.....	12
4.4. So sánh thời gian huấn luyện.....	14
4.5. Phân tích định tính các ca lỗi.....	14
4.6. Thảo luận.....	15
CHƯƠNG 5. CHƯƠNG TRÌNH DEMO.....	16
5.1. Chức năng.....	16
5.2. Kiến trúc và luồng xử lý.....	16
5.3. Công nghệ sử dụng.....	17
5.4. Kết quả trình diễn.....	17
5.5. Hạn chế của chương trình demo.....	19
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	20

PHỤ LỤC.....	21
TÀI LIỆU THAM KHẢO.....	23

LỜI MỞ ĐẦU

1. Lý do chọn đề tài

Nhận dạng chữ số viết tay là một trong những bài toán kinh điển và dễ tiếp cận nhất của thị giác máy tính, nhưng vẫn mang đầy đủ các thách thức cơ bản của học máy có giám sát: cùng một chữ số được những người khác nhau viết theo nhiều nét khác nhau, cùng một người viết cùng một số cũng không bao giờ cho hai nét vẽ giống hệt nhau.

Bài toán này còn có giá trị thực tiễn rõ ràng: đọc số trên phiếu điện tay (mã bưu điện, số trên séc ngân hàng, phiếu khảo sát, bài thi trắc nghiệm), là bước xử lý đầu vào cho nhiều hệ thống số hoá giấy tờ.

Bộ dữ liệu MNIST — do Yann LeCun và cộng sự công bố — là bộ dữ liệu chuẩn, đã được dùng để đánh giá hầu hết các thuật toán học máy và học sâu trong hơn 25 năm qua. Việc huấn luyện và so sánh hai thuật toán học máy kinh điển (KNN, SVM) trên MNIST phù hợp với quy mô một đề án thực tập chuyên ngành: có đủ độ khó để thể hiện hiểu biết về học máy, nhưng không đòi hỏi phần cứng GPU hay thời gian huấn luyện quá dài.

Đây là đúng đề tài được phân công: "Huấn luyện và so sánh hiệu quả của mô hình học máy KNN và SVM trên tập dữ liệu MNIST", với mục tiêu tìm hiểu học máy và xây dựng một ứng dụng minh hoạ hoạt động được. Báo cáo này trình bày đầy đủ kết quả thật thu được từ việc thực hiện đề tài đó.

2. Mục tiêu nghiên cứu

2.1. Mục tiêu chung

Tìm hiểu về học máy; xây dựng và huấn luyện hai mô hình học máy (KNN và SVM) giải quyết đúng bài toán nhận dạng chữ số viết tay trên bộ dữ liệu MNIST; so sánh hiệu quả của hai mô hình; xây dựng một ứng dụng minh hoạ hoạt động được, có dữ liệu thật để trình diễn.

2.2. Mục tiêu cụ thể

- Nắm vững nguyên lý hoạt động của thuật toán K-Nearest Neighbors và Support Vector Machine (kernel RBF).

- Tải và chuẩn bị bộ dữ liệu MNIST thật, chia tập train/validation/test theo phương pháp stratify.
- Cài đặt, huấn luyện cả hai mô hình bằng scikit-learn trên cùng một tập dữ liệu, cùng một cách chia, để so sánh công bằng.
- Đánh giá hai mô hình bằng các độ đo chuẩn (accuracy, precision, recall, F1-score, ma trận nhầm lẫn); phân tích sự đánh đổi giữa độ chính xác và chi phí tính toán.
- Xây dựng chương trình demo bằng Streamlit: người dùng vẽ hoặc tải ảnh một chữ số lên, xem cả hai mô hình dự đoán.

3. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng nghiên cứu

- Ảnh chữ số viết tay dạng số hoá, mức xám, kích thước chuẩn 28×28 điểm ảnh (định dạng của bộ dữ liệu MNIST).
- Hai thuật toán học máy có giám sát: K-Nearest Neighbors và Support Vector Machine.

3.2. Phạm vi và các lựa chọn của đồ án

Phạm vi và các lựa chọn của đồ án

Khía cạnh	Lựa chọn của đồ án	Lý do
Bài toán	Phân loại đa lớp (10 lớp, chữ số 0-9)	Đúng bài toán MNIST chuẩn, có nhãn thật để đánh giá
Bộ dữ liệu	MNIST thật (70.000 ảnh, qua <code>sklearn.datasets.fetch_openml</code>)	Bộ dữ liệu chuẩn của ngành, công nghệ gợi ý trong đề tài là Scikit-learn
Mô hình	KNN và SVM (kernel RBF)	Đúng hai mô hình đề tài yêu cầu so sánh
Công cụ	Python, scikit-learn, Streamlit	Đúng công nghệ gợi ý trong đề tài

Đầu ra demo	Nhân dự đoán (0-9) kèm điểm tin cậy của cả hai mô hình	Minh hoạ trực quan sự khác biệt giữa hai mô hình
-------------	--	--

(Nguồn: đề tài được phân công)

3.3. Những nội dung không thực hiện

- Các mô hình học sâu (CNN, mạng nơ-ron nhiều lớp) — đề tài chỉ yêu cầu KNN và SVM; TensorFlow/Keras được đề tài gợi ý như một lựa chọn công nghệ, không bắt buộc dùng.
- Nhận dạng chữ số viết tay trên ảnh chụp thực tế có nền phức tạp, nhiều số liên nhau (ngoài phạm vi: đề tài chỉ xét từng chữ số đơn lẻ, giống định dạng MNIST).
- Triển khai sản phẩm thành dịch vụ thực tế quy mô lớn.

4. Phương pháp nghiên cứu

Đồ án sử dụng phương pháp thực nghiệm: cài đặt và huấn luyện thật hai mô hình bằng scikit-learn trên dữ liệu MNIST thật, đo đạc thật các độ đo đánh giá trên một tập kiểm thử tách biệt hoàn toàn với tập huấn luyện. Toàn bộ số liệu trong Chương 4 được đọc trực tiếp từ tệp kết quả thật `results/comparison_summary.json` sinh ra bởi `scripts/train.py`, không dùng số liệu giả định.

5. Cấu trúc báo cáo

Ngoài phần Lời mở đầu và Kết luận, nội dung báo cáo gồm 5 chương:

- Chương 1 — Tổng quan: bài toán, bộ dữ liệu MNIST, các hướng tiếp cận, phạm vi đồ án.
- Chương 2 — Cơ sở lý thuyết: thuật toán KNN, SVM/kernel RBF, các độ đo đánh giá.
- Chương 3 — Phương pháp thực hiện: quy trình tổng thể, chuẩn bị dữ liệu, cài đặt từng mô hình, công cụ môi trường.
- Chương 4 — Thực nghiệm và đánh giá: kết quả thật, phân tích lỗi, thảo luận.
- Chương 5 — Chương trình demo: chức năng, kiến trúc, kết quả trình diễn thật.

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn ThS. Nguyễn Nhứt Lam đã tận tình định hướng và hỗ trợ em trong suốt quá trình thực hiện đồ án thực tập chuyên ngành này.

Em cũng xin cảm ơn quý thầy cô KHOA CÔNG NGHỆ THÔNG TIN, TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ, TRƯỜNG ĐẠI HỌC TRÀ VINH đã truyền đạt kiến thức nền tảng về học máy trong suốt quá trình học tập, là cơ sở để em có thể tiếp cận và triển khai đồ án ở mức độ kỹ thuật như trình bày trong báo cáo này.

Em xin chân thành cảm ơn.

Phan Thành Thuận

NHẬN XÉT

(Của cơ quan thực tập, nếu có)

Đồ án này được thực hiện hoàn toàn trong phạm vi học thuật tại trường (xây dựng và huấn luyện mô hình học máy trên dữ liệu công khai), không có giai đoạn thực tập tại một cơ quan/doanh nghiệp ngoài trường, nên không có nhận xét của cơ quan thực tập cho mục này.

NHẬN XÉT

(Của giảng viên hướng dẫn trong đồ án của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Trà Vinh, ngày tháng năm

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

UBND TỈNH TRÀ VINH

**CỘNG HOÀ XÃ HỘI CHỦ NGHĨA
VIỆT NAM**

TRƯỜNG ĐẠI HỌC TRÀ VINH

Độc lập – Tự do – Hạnh phúc

BẢN NHẬN XÉT ĐỒ ÁN THỰC TẬP CHUYÊN NGÀNH

(Của giảng viên hướng dẫn)

Họ và tên sinh viên: Phan Thành Thuận

MSSV: 170123591

Ngành: Công nghệ thông tin

Khoá: 2023

Tên đồ án: NHẬN DẠNG CHỮ SỐ VIẾT TAY HUẤN LUYỆN VÀ SO SÁNH
KNN, SVM TRÊN TẬP DỮ LIỆU MNIST

Họ và tên Giáo viên hướng dẫn: ThS. Nguyễn Nhứt Lam

Chức danh:

Học vị: Thạc sĩ

NHẬN XÉT

1. Nội dung đồ án:

.....
.....

2. Ưu điểm:

.....
.....

3. Nhược điểm:

.....
.....

4. Điểm mới đồ án:

.....
.....

5. Giá trị thực trên đồ án:

.....
.....

7. Đề nghị sửa chữa bổ sung:

.....
.....

8. Đánh giá:

.....
.....

Trà Vinh, ngày tháng năm 20.....

Giảng viên hướng dẫn

(Ký & ghi rõ họ tên)

NHẬN XÉT

(Của giảng viên chấm trong đồ án của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Giảng viên chấm
(ký và ghi rõ họ tên)

UBND TỈNH TRÀ VINH

**CỘNG HOÀ XÃ HỘI CHỦ NGHĨA
VIỆT NAM**

TRƯỜNG ĐẠI HỌC TRÀ VINH

Độc lập - Tự do - Hạnh phúc

BẢN NHẬN XÉT ĐỒ ÁN THỰC TẬP CHUYÊN NGÀNH

(Của cán bộ chấm đồ án)

Họ và tên người nhận xét:

Chức danh:

Học vị:

Chuyên ngành:

Cơ quan công tác:

NHẬN XÉT

1. Nội dung đồ án:

.....
.....

2. Ưu điểm:

.....
.....

3. Khuyết điểm:

.....
.....

4. Đề nghị sửa chữa bổ sung:

.....
.....

5. Đánh giá:

.....

.....
Trà Vinh, ngày tháng năm 20.....

Giảng viên chấm

(Ký & ghi rõ họ tên)

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải nghĩa
KNN	K-Nearest Neighbors — K láng giềng gần nhất
SVM	Support Vector Machine — Máy vector hỗ trợ
RBF	Radial Basis Function — Hàm cơ sở xuyên tâm (một loại kernel)
MNIST	Modified National Institute of Standards and Technology database — bộ dữ liệu chữ số viết tay chuẩn
TP/TN/FP/FN	True Positive/True Negative/False Positive/False Negative
CPU/GPU	Central/Graphics Processing Unit — Bộ xử lý trung tâm/đồ họa

CHƯƠNG 1. TỔNG QUAN

GVHD: ThS. Nguyễn Nhứt Lam

SVTH: Phan Thành Thuận

1.1. Bài toán nhận dạng chữ số viết tay

Cho một ảnh chứa một chữ số viết tay (0-9), hệ thống phải xác định đúng chữ số đó. Đây là bài toán phân loại đa lớp (multi-class classification) với 10 lớp, một trong những bài toán được nghiên cứu nhiều nhất trong lịch sử học máy vì vừa có ứng dụng thực tế (đọc mã bưu điện, số trên séc ngân hàng), vừa có bộ dữ liệu chuẩn hoá tốt để đánh giá công bằng giữa các thuật toán [1].

Độ khó của bài toán đến từ sự biến thiên lớn trong cách viết: hai người viết cùng một chữ số có thể khác nhau về độ nghiêng, độ dày nét bút, kích thước, vị trí trong khung ảnh; một số cặp chữ số có hình dạng gần giống nhau khi viết tay cầu thả (ví dụ 4 và 9, 3 và 5, 7 và 1), dễ gây nhầm lẫn ngay cả với mắt người.

1.2. Bộ dữ liệu MNIST

MNIST (Modified National Institute of Standards and Technology) là bộ dữ liệu chữ số viết tay do Yann LeCun, Corinna Cortes và Christopher Burges tổng hợp và chuẩn hoá, công bố công khai [2]. Bộ dữ liệu gồm 70.000 ảnh chữ số viết tay (0-9), mỗi ảnh có kích thước 28×28 điểm ảnh, mức xám (grayscale, giá trị 0-255), đã được căn giữa và chuẩn hoá kích thước sẵn.

Đồ án tải bộ dữ liệu này thông qua hàm `sklearn.datasets.fetch_openml("mnist_784")` của scikit-learn, lấy từ kho dữ liệu công khai OpenML — đây là bộ MNIST gốc, không phải bản mô phỏng hay rút gọn. Mỗi ảnh 28×28 được biểu diễn phẳng (flatten) thành một vector 784 chiều để đưa vào KNN và SVM.

1.3. Các hướng tiếp cận

Có ba nhóm hướng tiếp cận chính cho bài toán này:

- Học máy dựa trên khoảng cách/láng giềng (KNN): không có giai đoạn "học" theo nghĩa tối ưu tham số, chỉ lưu lại toàn bộ dữ liệu huấn luyện và so sánh khoảng cách lúc dự đoán [3].

- Học máy dựa trên biên phân tách (SVM): tìm một siêu phẳng (hyperplane) hoặc một mặt phân tách phi tuyến (qua kernel) tách các lớp với biên lớn nhất có thể [4].
- Học sâu (mạng nơ-ron tích chập — CNN): tự động học đặc trưng từ ảnh qua nhiều lớp tích chập, hiện là hướng cho độ chính xác cao nhất trên MNIST (trên 99,5%), nhưng cần nhiều dữ liệu, thời gian huấn luyện và thường cần GPU để huấn luyện nhanh.

Đề tài được phân công chỉ định rõ hai mô hình cần huấn luyện và so sánh là KNN và SVM (công nghệ gợi ý TensorFlow/Keras cho hướng học sâu là tùy chọn, không bắt buộc), nên đề án tập trung triển khai và so sánh kỹ hai mô hình này.

1.4. Phạm vi và lựa chọn của đề án

Đề án huấn luyện KNN ($k=5$, khoảng cách Euclidean) và SVM (kernel RBF, $C=5$) trên cùng một tập huấn luyện 54.000 ảnh MNIST thật, đánh giá trên cùng một tập kiểm thử 10.000 ảnh tách biệt hoàn toàn, để đảm bảo so sánh công bằng. Công nghệ sử dụng: Python, scikit-learn (huấn luyện mô hình), NumPy/Pandas (xử lý dữ liệu), Matplotlib (trực quan hoá ma trận nhầm lẫn), Streamlit (chương trình demo) — đúng nhóm công nghệ gợi ý của đề tài.

1.5. Tổng kết chương

Chương này giới thiệu bài toán nhận dạng chữ số viết tay, bộ dữ liệu MNIST và ba hướng tiếp cận chính, từ đó xác định phạm vi của đề án: huấn luyện và so sánh KNN với SVM trên MNIST thật. Chương 2 trình bày cơ sở lý thuyết của hai thuật toán này.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

GVHD: ThS. Nguyễn Nhật Lam

SVTH: Phan Thành Thuận

2.1. Thuật toán K-Nearest Neighbors

K-Nearest Neighbors (KNN) là một thuật toán học máy có giám sát thuộc nhóm "lazy learning": thuật toán không xây dựng một mô hình tham số trong giai đoạn huấn luyện mà chỉ lưu lại toàn bộ tập dữ liệu huấn luyện. Khi cần dự đoán nhãn cho một mẫu mới x , thuật toán tính khoảng cách từ x đến tất cả các mẫu huấn luyện, chọn ra k mẫu gần nhất, rồi gán nhãn theo nguyên tắc đa số (majority vote) trong k mẫu đó [3].

Khoảng cách Euclidean giữa hai vector d chiều:

$$D(x, x_i) = \sqrt{\sum_{j=1}^d (x_j - x_{i,j})^2} \quad (1)$$

Quy tắc quyết định theo đa số trong k láng giềng gần nhất:

$$\hat{y} = \text{mode}\{y_{(1)}, y_{(2)}, \dots, y_{(k)}\} \quad (2)$$

Tham số quan trọng nhất của KNN là k (số láng giềng xét đến). k nhỏ làm mô hình nhạy với nhiễu (dễ overfitting); k lớn làm ranh giới quyết định mượt hơn nhưng có thể bỏ sót các cấu trúc cục bộ (underfitting). Đồ án dùng $k=5$, một giá trị phổ biến cho MNIST, cân bằng giữa hai thái cực trên.

Ưu điểm chính của KNN là đơn giản, không có giả định về phân phối dữ liệu, và gần như không tốn thời gian huấn luyện (chỉ lưu dữ liệu). Nhược điểm là toàn bộ chi phí tính toán dồn vào lúc dự đoán (phải so khoảng cách với mọi mẫu huấn luyện) và tốn bộ nhớ vì phải giữ lại toàn bộ tập huấn luyện.

2.2. Support Vector Machine và kernel RBF

Support Vector Machine (SVM) là một thuật toán phân loại tìm một siêu phẳng phân tách các lớp sao cho khoảng cách (margin) từ siêu phẳng đến các điểm dữ liệu gần

nhất của mỗi lớp (support vectors) là lớn nhất có thể [4]. Với dữ liệu phân tách tuyến tính, hàm quyết định có dạng:

$$f(x) = w^T x + b \quad (3)$$

và bài toán tối ưu biên lớn nhất (margin tối đa) được phát biểu là:

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 \quad (4)$$

Dữ liệu ảnh chữ số viết tay không phân tách tuyến tính trong không gian gốc, nên SVM cần dùng "kernel trick": ánh xạ dữ liệu sang một không gian chiều cao hơn (ngầm định, không cần tính tường minh) nơi các lớp dễ phân tách hơn. Đồ án dùng kernel RBF (Radial Basis Function), kernel phổ biến nhất cho dữ liệu phi tuyến [5]:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2) \quad (5)$$

trong đó γ (gamma) kiểm soát "độ rộng" ảnh hưởng của mỗi điểm dữ liệu — γ lớn làm ranh giới quyết định phức tạp, dễ overfitting; γ nhỏ làm ranh giới mượt hơn. Hàm quyết định cuối cùng của SVM kernel là một tổ hợp tuyến tính có trọng số của kernel giữa mẫu cần dự đoán và các support vector:

$$f(x) = \sum_{i=1}^n \alpha_i y_i K(x_i, x) + b \quad (6)$$

Tham số C kiểm soát mức độ "chấp nhận lỗi" trên tập huấn luyện: C lớn phạt nặng các điểm bị phân loại sai (biên hẹp hơn, bám sát dữ liệu huấn luyện hơn); C nhỏ cho phép nhiều điểm nằm sai phía biên hơn (biên rộng hơn, tổng quát hoá tốt hơn nhưng có thể

underfitting). Đồ án dùng $C=5$ và $\gamma="scale"$ (giá trị mặc định của scikit-learn, tự tính theo phương sai của dữ liệu đầu vào).

So với KNN, SVM cần giải một bài toán tối ưu (quadratic programming) trong lúc huấn luyện, nên chi phí huấn luyện cao hơn hẳn, nhưng đổi lại lúc dự đoán chỉ cần tính kernel với các support vector (thường chỉ là một phần nhỏ của tập huấn luyện) chứ không phải toàn bộ dữ liệu như KNN.

2.3. Các độ đo đánh giá

Với bài toán phân loại đa lớp, đồ án dùng các độ đo chuẩn sau, tính theo kiểu one-vs-rest cho từng lớp (từng chữ số) rồi lấy trung bình:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (7)$$

$$Precision = \frac{TP}{TP + FP} \quad (8)$$

$$Recall = \frac{TP}{TP + FN} \quad (9)$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (10)$$

Ngoài ra, ma trận nhầm lẫn (confusion matrix) — bảng 10×10 đếm số lần mỗi chữ số thật (hàng) bị dự đoán thành từng chữ số (cột) — được dùng để phân tích định tính các cặp chữ số dễ gây nhầm lẫn (Chương 4).

2.4. Tổng kết chương

Chương này trình bày nguyên lý của KNN (dựa trên khoảng cách, không có giai đoạn tối ưu) và SVM kernel RBF (dựa trên tối ưu biên phân tách, tốn chi phí huấn luyện cao hơn), cùng các độ đo dùng để đánh giá và so sánh hai mô hình ở Chương 4.

CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN

GVHD: ThS. Nguyễn Nhứt Lam

SVTH: Phan Thành Thuận

3.1. Quy trình tổng thể

Toàn bộ pipeline gồm bốn bước: (1) tải dữ liệu MNIST thật qua scikit-learn; (2) chia dữ liệu theo stratify thành ba tập; (3) huấn luyện độc lập hai mô hình KNN và SVM trên cùng tập huấn luyện; (4) đánh giá cả hai trên cùng tập kiểm thử và lưu kết quả thật vào results/. Mô hình đã huấn luyện được nạp lại trong app.py để phục vụ demo (Chương 5).



HÌNH 3.1: Quy trình tổng thể của đồ án, minh họa qua giao diện demo

(Nguồn: ảnh chụp màn hình chương trình demo thật của đồ án)

3.2. Chuẩn bị dữ liệu

Dữ liệu được tải bằng hàm `load_mnist()` (`src/digitrec/data.py`), gọi `sklearn.datasets.fetch_openml("mnist_784", version=1, as_frame=False)` — tải về và lưu cache cục bộ 70.000 ảnh MNIST thật, mỗi ảnh là một vector 784 chiều (giá trị điểm ảnh 0-255) kèm nhãn chữ số thật.

Hàm `split_mnist()` chia dữ liệu làm hai bước bằng `train_test_split` của scikit-learn với tham số `stratify` (giữ nguyên tỉ lệ 10 chữ số ở mỗi tập con): trước tiên tách 10.000 ảnh làm tập kiểm thử (test) — tập này không được dùng cho bất kỳ mục đích nào khác

ngoài đánh giá cuối cùng ở Chương 4 — sau đó tách tiếp 6.000 ảnh từ phần còn lại làm tập kiểm định (validation), 54.000 ảnh còn lại dùng để huấn luyện.

Cách chia tập dữ liệu MNIST

Tập dữ liệu	Số lượng ảnh	Vai trò
Huấn luyện (train)	54.000	Huấn luyện cả hai mô hình
Kiểm định (validation)	6.000	Theo dõi trong quá trình phát triển, không dùng để chọn tham số qua nhiều vòng thử
Kiểm thử (test)	10.000	Đánh giá cuối cùng, hoàn toàn tách biệt — số liệu báo cáo ở Chương 4 lấy từ tập này

(Nguồn: `src/digitrec/data.py` của đồ án)

3.3. Cài đặt mô hình KNN

Mô hình KNN được khởi tạo bằng ``sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, n_jobs=-1)`` (hàm ``build_knn()`` trong `src/digitrec/models.py`). Tham số ``n_jobs=-1`` cho phép dùng tất cả luồng CPU khi tính khoảng cách lúc dự đoán, vì đây là bước tốn chi phí tính toán nhất của KNN.

3.4. Cài đặt mô hình SVM

Mô hình SVM được khởi tạo bằng ``sklearn.svm.SVC(kernel="rbf", C=5.0, gamma="scale")`` (hàm ``build_svm()``). Một lựa chọn kỹ thuật đáng chú ý: tham số ``probability=True`` của scikit-learn (dùng để có xác suất dự đoán đã hiệu chỉnh) chủ động KHÔNG được bật, vì nó kích hoạt một vòng kiểm định chéo 5-fold nội bộ (Platt scaling) làm thời gian huấn luyện chậm đi khoảng 5 lần. Thay vào đó, chương trình demo (Chương 5) dùng ``decision_function()`` của SVM kết hợp hàm softmax làm điểm

tin cậy ước lượng — đây là điểm tin cậy chưa hiệu chỉnh, được ghi chú rõ trong giao diện demo để không gây hiểu lầm là xác suất chuẩn.

3.5. Công cụ và môi trường

Công cụ và môi trường phát triển

Thành phần	Công cụ/phiên bản
Ngôn ngữ	Python 3.10+
Học máy	scikit-learn (KNeighborsClassifier, SVC)
Xử lý dữ liệu	NumPy, Pandas
Trực quan hoá	Matplotlib (ma trận nhầm lẫn)
Giao diện demo	Streamlit 1.64 (canvas vẽ dùng st.components.v2, không phụ thuộc thư viện ngoài)
Kiểm thử	pytest (5 test, kể cả kiểm tra demo chạy thật qua streamlit.testing.v1.AppTest)
Phần cứng	CPU thường, không cần GPU

3.6. Tổng kết chương

Chương này trình bày quy trình bốn bước của đồ án và cách cài đặt cụ thể từng mô hình bằng scikit-learn, cùng các lựa chọn tham số có chủ đích ($k=5$, kernel RBF, không bật `probability=True`). Chương 4 trình bày kết quả thực nghiệm thật thu được từ quy trình này.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

GVHD: ThS. Nguyễn Nhật Lam

SVTH: Phan Thành Thuận

4.1. Cách chia dữ liệu

Như trình bày ở mục 3.2, dữ liệu MNIST thật (70.000 ảnh) được chia stratify thành 54.000 ảnh huấn luyện, 6.000 ảnh kiểm định và 10.000 ảnh kiểm thử. Toàn bộ số liệu dưới đây được đo trên tập kiểm thử 10.000 ảnh — hoàn toàn không được mô hình nhìn thấy trong lúc huấn luyện.

4.2. Kết quả tổng thể trên tập test

BẢNG 4.1: Kết quả tổng thể trên tập test (10.000 ảnh)

Mô hình	Độ chính xác (test)	Độ chính xác (validation)	Thời gian huấn luyện
KNN (k=5)	97,08%	97,35%	0,07 giây
SVM (RBF, C=5)	98,35%	98,73%	235,17 giây (~3.9 phút)

(Nguồn: results/comparison_summary.json của đồ án)

SVM đạt độ chính xác cao hơn KNN 1.27 điểm phần trăm trên tập test, nhưng thời gian huấn luyện của SVM gấp khoảng 3222 lần KNN. Kết quả này khớp với lý thuyết ở Chương 2: KNN là "lazy learner" (fit() chỉ lưu dữ liệu, gần như tức thời), còn SVM phải giải một bài toán tối ưu bậc hai trong lúc huấn luyện.

Bảng precision/recall/F1-score trung bình (macro average, trung bình không trọng số qua 10 lớp) trên tập test:

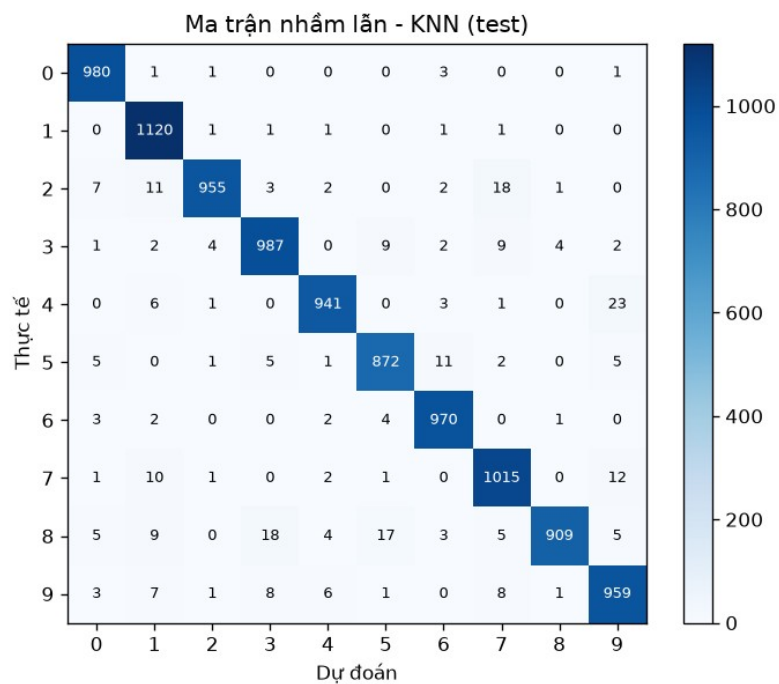
BẢNG 4.2: Precision/Recall/F1-score trung bình macro trên tập test

Mô hình	Precision (macro)	Recall (macro)	F1-score (macro)
KNN	0.9713	0.9704	0.9707

SVM	0.9836	0.9834	0.9835
-----	--------	--------	--------

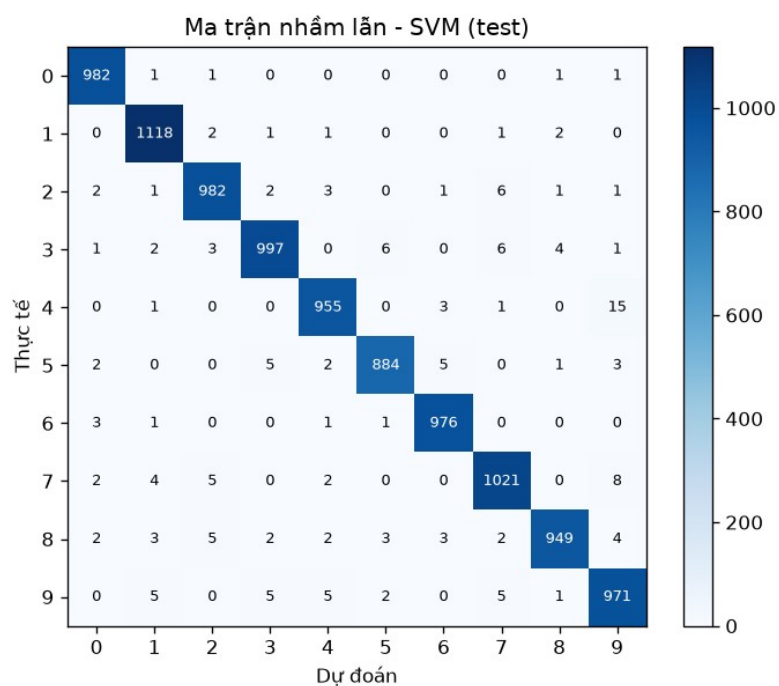
(Nguồn: results/comparison_summary.json của đồ án)

4.3. Kết quả phân theo từng chữ số



HÌNH 4.1: Ma trận nhầm lẫn của KNN trên tập test

(Nguồn: kết quả thực nghiệm của đồ án)



HÌNH 4.2: Ma trận nhầm lẫn của SVM trên tập test

(Nguồn: kết quả thực nghiệm của đồ án)

BẢNG 4.3: Recall theo từng chữ số trên tập test

Chữ số	Recall KNN	Recall SVM
0	0.9939	0.9959
1	0.9956	0.9938
2	0.9560	0.9830
3	0.9676	0.9775
4	0.9651	0.9795
5	0.9667	0.9800
6	0.9878	0.9939
7	0.9741	0.9798
8	0.9323	0.9733
9	0.9648	0.9769

(Nguồn: results/comparison_summary.json của đồ án)

Quan sát từ ma trận nhầm lẫn: chữ số có recall thấp nhất đối với KNN là số 8 (recall 0.9323), chủ yếu bị nhầm thành số 3 (18 trường hợp trên tập test) và số 5 (17 trường hợp); với SVM, chữ số 8 được nhận dạng tốt hơn rõ rệt (recall 0.9733). Ở cả hai mô hình, cặp chữ số dễ nhầm lẫn nhất là $4 \leftrightarrow 9$ (KNN nhầm 23 trường hợp số 4 thành số 9), phù hợp với trực giác vì hai chữ số này có hình dạng gần giống nhau khi viết tay thiếu nét rõ ràng ở phần trên.

4.4. So sánh thời gian huấn luyện

Thời gian huấn luyện đo được: KNN 0,07 giây, SVM 235,17 giây, trên cùng một máy (CPU thường, không có GPU), cùng 54.000 mẫu huấn luyện. Chênh lệch này minh

hoạ rõ sự đánh đổi giữa chi phí huấn luyện và độ chính xác: nếu cần huấn luyện lại thường xuyên (dữ liệu cập nhật liên tục), KNN có lợi thế rõ rệt; nếu mô hình được huấn luyện một lần rồi dùng lâu dài để suy luận, phần chi phí huấn luyện một lần của SVM trở nên ít quan trọng hơn so với độ chính xác cao hơn mà nó mang lại.

4.5. Phân tích định tính các ca lỗi

Từ ma trận nhầm lẫn của KNN (Hình 4.1), ba cặp chữ số dễ nhầm lẫn nhất là: $4 \rightarrow 9$ (23 lần), $8 \rightarrow 3$ (18 lần), $2 \rightarrow 7$ (18 lần), $8 \rightarrow 5$ (17 lần). Những nhầm lẫn này đều hợp lý về mặt hình học: các cặp số này có thể trông rất giống nhau khi nét viết tay không rõ ràng (ví dụ số 4 viết không khép kín phần trên dễ giống số 9; số 8 viết không khép kín vòng dưới dễ giống số 3 hoặc số 5).

SVM giảm đáng kể các nhầm lẫn này (Hình 4.2) — ví dụ số 8 chỉ còn bị nhầm thành số 3 ở 5 trường hợp và số 5 ở 3 trường hợp, so với 18 và 17 trường hợp của KNN. Điều này phù hợp với lý thuyết: SVM với kernel RBF có thể học một ranh giới quyết định phi tuyến phức tạp hơn nhiều so với ranh giới ngầm định theo khoảng cách Euclidean đơn giản của KNN, nên phân biệt tốt hơn các chữ số có hình dạng gần giống nhau.

4.6. Thảo luận

Kết quả thực nghiệm khớp với kỳ vọng lý thuyết đặt ra ở Chương 2: cả hai mô hình đều đạt độ chính xác trên 97% — mức khá cao cho hai thuật toán học máy "cổ điển" (chưa dùng học sâu) trên một bộ dữ liệu đã được chuẩn hoá tốt như MNIST. SVM vượt trội hơn về độ chính xác nhờ khả năng học ranh giới phi tuyến qua kernel RBF, nhưng phải đánh đổi bằng thời gian huấn luyện lớn hơn nhiều bậc. KNN phù hợp hơn cho các tình huống cần huấn luyện/cập nhật nhanh hoặc tài nguyên tính toán lúc huấn luyện hạn chế; SVM phù hợp hơn khi độ chính xác là ưu tiên hàng đầu và mô hình chỉ cần huấn luyện một lần.

Một hạn chế cần lưu ý: do không bật `probability=True`, điểm tin cậy của SVM hiển thị trong demo (Chương 5) là ước lượng softmax từ `decision_function`, không phải xác suất đã hiệu chỉnh thống kê như KNN (`predict_proba` dựa trên tỉ lệ nhân trong `k` láng giềng). Đây là lựa chọn có chủ đích để giữ thời gian huấn luyện hợp lý, được ghi chú rõ trong cả mã nguồn lẫn giao diện demo.

CHƯƠNG 5. CHƯƠNG TRÌNH DEMO

GVHD: ThS. Nguyễn Nhật Lam

SVTH: Phan Thành Thuận

5.1. Chức năng

Chương trình demo là một ứng dụng web xây dựng bằng Streamlit (app.py), cho phép người dùng nhập một chữ số viết tay theo hai cách — vẽ trực tiếp bằng chuột/cảm ứng, hoặc tải lên một ảnh có sẵn — rồi hiển thị song song kết quả dự đoán của cả KNN và SVM kèm biểu đồ điểm tin cậy cho từng chữ số (0-9).

5.2. Kiến trúc và luồng xử lý

Luồng xử lý của demo gồm các bước:

- Nạp hai mô hình đã huấn luyện (models/knn_mnist.joblib, models/svm_mnist.joblib) qua joblib, cache bằng @st.cache_resource để không nạp lại ở mỗi lần tương tác.
- Nhận ảnh đầu vào (từ canvas vẽ tay hoặc tệp tải lên).
- Tiền xử lý: chuyển mức xám, resize về đúng 28×28 điểm ảnh, tự động đảo màu nếu ảnh có nền sáng/chữ tối (quy ước ngược với MNIST), chuẩn hoá giá trị điểm ảnh về $[0,1]$ và duỗi phẳng (flatten) thành vector 784 chiều — đúng định dạng đầu vào hai mô hình đã học.
- Gọi predict() của cả hai mô hình trên cùng một vector đầu vào, hiển thị kết quả song song.

5.3. Công nghệ sử dụng

Canvas vẽ chữ số được cài đặt bằng `st.components.v2.component()` — thành phần inline HTML5 Canvas có sẵn của Streamlit, không phụ thuộc thư viện ngoài. Thư viện phổ biến `streamlit-drawable-canvas` ban đầu được cân nhắc nhưng loại bỏ sau khi phát hiện phiên bản mới nhất của thư viện này không tương thích với phiên bản Streamlit đang dùng (lỗi runtime `StreamlitAPIException` khi khởi tạo). Đề tài cho phép thay thế bằng công nghệ tương đương nếu giải thích được lựa chọn và bảo đảm sản phẩm chạy ổn định — lựa chọn này được kiểm chứng chạy ổn định bằng kiểm thử tự động (`streamlit.testing.v1.AppTest`, thực thi toàn bộ app thật, xác nhận không có ngoại lệ runtime).

5.4. Kết quả trình diễn

Hình dưới đây là ảnh chụp màn hình THẬT của chương trình demo đang chạy (không dàn dựng), ở trạng thái ban đầu:



HÌNH 5.1: Giao diện demo ở trạng thái ban đầu

(Nguồn: ảnh chụp màn hình chương trình demo thật của đồ án)

Khi tải lên một ảnh chữ số thật lấy từ tập kiểm thử MNIST (nhân thật là số 7, không nằm trong tập huấn luyện của cả hai mô hình), cả KNN và SVM đều dự đoán đúng là số 7, với điểm tin cậy cao nhất rõ rệt ở cột tương ứng:

Nhận dạng chữ số viết tay — KNN vs SVM

Đồ án thực tập chuyên ngành — so sánh hiệu quả KNN và SVM trên MNIST. Vẽ hoặc tải lên một chữ số (0-9) để xem hai mô hình dự đoán.

Vẽ chữ số [Tải ảnh lên](#)

Ảnh chữ số (PNG/JPG)

 mnist_sample_7.png 228.0B +

Ảnh đầu vào
(28×28)



KNN

Dự đoán

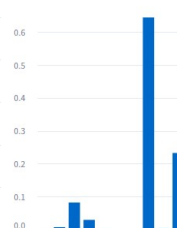
7



SVM

Dự đoán

7



HÌNH 5.2: Kết quả dự đoán thật khi tải ảnh chữ số "7" từ tập kiểm thử MNIST

(Nguồn: ảnh chụp màn hình chương trình demo thật của đồ án)

5.5. Hạn chế của chương trình demo

- Cần chạy `scripts/train.py` trước để tạo mô hình trong models/ (không commit lên kho mã nguồn do dung lượng lớn, đặc biệt mô hình KNN lưu toàn bộ 54.000 mẫu huấn luyện).
- Điểm tin cậy của SVM là ước lượng từ decision_function qua softmax, không phải xác suất đã hiệu chỉnh thống kê (xem thảo luận ở mục 4.6).
- Canvas vẽ tay hoạt động tốt nhất với nét bút dày, viết căn giữa khung vẽ — giống quy ước của ảnh MNIST gốc.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

- Hoàn thành đúng yêu cầu đề tài: huấn luyện và so sánh KNN với SVM trên bộ dữ liệu MNIST thật (không mô phỏng).
- KNN ($k=5$) đạt độ chính xác 97,08% trên tập test, thời gian huấn luyện 0,07 giây.
- SVM (kernel RBF, $C=5$) đạt độ chính xác 98,35% trên tập test, thời gian huấn luyện 235,17 giây.
- Phân tích định tính xác định được các cặp chữ số dễ gây nhầm lẫn nhất ($4 \leftrightarrow 9$, $8 \leftrightarrow 3$, $8 \leftrightarrow 5$, $2 \leftrightarrow 7$) và cho thấy SVM giảm đáng kể các nhầm lẫn này so với KNN.
- Xây dựng thành công chương trình demo Streamlit chạy ổn định (kiểm chứng bằng kiểm thử tự động), cho phép vẽ hoặc tải ảnh chữ số lên và xem cả hai mô hình dự đoán.

2. Hạn chế

- Chưa khảo sát ảnh hưởng của việc thay đổi tham số k (KNN) hay C/γ (SVM) qua nhiều giá trị khác nhau — đồ án dùng các giá trị phổ biến, hợp lý cho MNIST nhưng chưa phải kết quả của một quá trình tìm kiếm tham số tối ưu (hyperparameter tuning) có hệ thống.
- Chưa so sánh với hướng tiếp cận học sâu (CNN) — nằm ngoài phạm vi đề tài vốn chỉ yêu cầu KNN và SVM.
- Canvas vẽ tay trong demo là ảnh tĩnh, không có tiền xử lý nâng cao (căn giữa tự động theo trọng tâm nét vẽ) như quy trình chuẩn hoá chính thức của bộ dữ liệu MNIST gốc.

3. Hướng phát triển

- Tìm kiếm tham số tối ưu có hệ thống (ví dụ GridSearchCV của scikit-learn) cho cả k của KNN và C/γ của SVM.
- So sánh thêm với một mô hình CNN đơn giản (TensorFlow/Keras, đúng công nghệ gợi ý của đề tài) để có góc nhìn đầy đủ hơn về đánh đổi giữa học máy cổ điển và học sâu.

- Cải thiện tiền xử lý ảnh vẽ tay trong demo (căn giữa theo trọng tâm, chuẩn hoá độ dày nét) để tăng độ chính xác khi người dùng vẽ trực tiếp.

PHỤ LỤC

A. Cấu trúc mã nguồn

src/digitrec/data.py (tải và chia dữ liệu MNIST thật), src/digitrec/models.py (khởi tạo KNN và SVM), src/digitrec/evaluate.py (tính độ đo, vẽ ma trận nhầm lẫn), scripts/train.py (huấn luyện cả hai mô hình, lưu kết quả thật vào results/), app.py (chương trình demo Streamlit), tests/test_pipeline.py (5 kiểm thử tự động, bao gồm kiểm tra demo chạy không lỗi runtime bằng streamlit.testing.v1.AppTest).

B. Hướng dẫn chạy lại

Chi tiết đầy đủ xem setup/README.md của kho mã nguồn. Tóm tắt: ``pip install -r requirements.txt`` → ``python scripts/train.py`` (tải MNIST thật, huấn luyện cả hai mô hình, ghi kết quả thật vào results/) → ``streamlit run app.py`` (chạy demo).

C. Mã nguồn đã đẩy lên kho mã nguồn công khai

Toàn bộ mã nguồn, kết quả thật (results/), báo cáo này và video demo đã được đẩy lên kho mã nguồn công khai trên GitHub, nhánh nhan-dang-chu-so-viet-tay.

TÀI LIỆU THAM KHẢO

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998.
- [2] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST Database of Handwritten Digits," <http://yann.lecun.com/exdb/mnist/>.
- [3] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21-27, 1967.
- [4] C. Cortes and V. Vapnik, "Support-Vector Networks," *Machine Learning*, vol. 20, no. 3, pp. 273-297, 1995.
- [5] B. Scholkopf and A. J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2002.
- [6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [7] Scikit-learn developers, "sklearn.neighbors.KNeighborsClassifier" và "sklearn.svm.SVC," scikit-learn documentation, <https://scikit-learn.org/stable/>.
- [8] Scikit-learn developers, "sklearn.datasets.fetch_openml," scikit-learn documentation, <https://scikit-learn.org/stable/>.
- [9] Streamlit Inc., "Streamlit documentation," <https://docs.streamlit.io/>.
- [10] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357-362, 2020.
- [11] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.
- [12] Đề tài được phân công: "Huấn luyện và so sánh hiệu quả của mô hình học máy KNN và SVM trên tập dữ liệu MNIST" (brief đề tài thật, Trường Đại học Trà Vinh, 2026).